

La Interfaz, el Contrato Secreto de tu Código

1. El Concepto: "¿Qué es una Interfaz?"

Olvida el código por un segundo. Piensa en una **interfaz** como un **contrato** o un **manual de usuario**.

Imagina un manual para un "Dispositivo de Audio". Este manual te garantiza que el dispositivo tendrá ciertos botones:

- `encender()`
- `apagar()`
- `subirVolumen()`

El manual es la **interfaz**. Define **QUÉ** se puede hacer.

Ahora, puedes tener muchos dispositivos que "firman" este contrato:

1. Un `RadioDeAuto` (implementa `encender()` para arrancar el coche).
2. Un `EquipoDeSonido` (implementa `encender()` con un botón físico).
3. Un `AsistenteDeVoz` (implementa `encender()` escuchando un comando).

Todos son diferentes (**Radio**, **Equipo**, **Voz**), pero tú, como usuario, sabes que si algo "firma" el contrato "DispositivoDeAudio", puedes **subirVolumen()**.

En POO (Programación Orientada a Objetos), una **Interfaz** es exactamente eso: un contrato que define **QUÉ** métodos debe tener una clase, pero **NUNCA** dice **CÓMO** deben funcionar.

El Mantra: "Define el QUÉ, no el CÓMO".

2. Nuestro Ejemplo Central: El Login 2FA

En nuestro chat, queríamos un sistema de Login 2FA. El problema era:

- **LoginManager** necesita validar un segundo factor.
- Pero el segundo factor podría ser un **ProveedorSms**, un **ProveedorTotp** (Authenticator) o un **ProveedorEmail**.

El **LoginManager** no quiere (¡no debe!) tener un **if** gigante para saber a quién llamar. Solo quiere un "botón" estándar que pueda presionar.

Aquí es donde creamos nuestro contrato, la **interface**.

3. El Poder Oculto: Interfaces y Polimorfismo

Lo que viste en el `LoginManager` se llama **Polimorfismo**.

- **Poli** = Múltiples
- **Morfismo** = Formas

Es la capacidad de tratar a objetos de diferentes clases (formas) de una manera uniforme, siempre y cuando todos "firmen" el mismo contrato (la misma interfaz).

El método `completarLogin` es polimórfico. Acepta **múltiples formas** de `ProveedorDosPasos`.

PHP-Interfaces con Login2FA guiado

Usar el login 2FA (Autenticación de Dos Factores) es un ejemplo excelente para explicar las interfaces.

Pensemos en el problema: tu sistema de login principal (la clase `LoginManager`, por ejemplo) no quiere saber *cómo* funciona el 2FA. No le importa si es un SMS, un Email, o Google Authenticator (TOTP).

El **LoginManager** solo quiere poder decirle a un objeto: "Oye, válida a este usuario".

Aquí es donde entra la **interface**. Una interfaz es un **contrato** que define qué métodos *debe* tener una clase, pero no cómo los implementa.

Empecemos por definir ese contrato. Lo llamaremos **ProveedorDosPasos**.

PHP

```
<?php
```

```
/**
 * Esta es la INTERFAZ (El Contrato).
 * Define QUÉ debe hacer un proveedor de 2FA, pero no CÓMO.
 */
interface ProveedorDosPasos
{
    /**
     * Inicia el desafío para el usuario (ej. enviar un SMS, generar un código).
     *
     * @param Usuario $usuario El usuario que intenta iniciar sesión.
     * @return bool True si el desafío se envió con éxito.
     */
    public function enviarDesafio(Usuario $usuario): bool;

    /**
     * Verifica si el código proporcionado por el usuario es válido.
     *
     * @param Usuario $usuario El usuario.
     * @param string $codigo El código 2FA (ej. "123456").
     * @return bool True si el código es válido.
     */
    public function verificarCodigo(Usuario $usuario, string $codigo): bool;
}

// --- Clases auxiliares solo para el ejemplo ---
class Usuario
{
    public function __construct(
        public string $nombre,
        public string $telefono,
        public string $secretoTotp
    ) {}
}
?>
```

Analicemos esto:

1. Usamos la palabra clave `interface` en lugar de `class`.
2. Los métodos `enviarDesafio` y `verificarCodigo` no tienen llaves `{}`. No tienen implementación, solo definen la "firma" (el nombre, los parámetros que aceptan y lo que devuelven).

Ahora tenemos el contrato. El siguiente paso es crear clases "concretas" que *cumplan* con este contrato.

¿Cómo empezarías a escribir la clase `ProveedorSms` que *implemente* esta interfaz? (Pista: ¿qué palabra clave de PHP usamos para "firmar" el contrato?)

La palabra clave que necesitamos para "firmar" el contrato (la interfaz) es `implements`.

Así es como empezaríamos:

PHP

```
<?php
```

```
// Incluimos la interfaz y la clase Usuario para que este archivo las conozca
```

```
// (Asumiendo que están en otros archivos)
```

```
// require_once 'ProveedorDosPasos.php';
```

```
// require_once 'Usuario.php';
```

```
/**
```

```
 * Esta es la implementación CONCRETA para enviar códigos 2FA por SMS.
```

```
 * "Implementa" el contrato ProveedorDosPasos.
```

```
 */
```

```
class ProveedorSms implements ProveedorDosPasos
```

```

{
    /**
     * Inicia el desafío para el usuario (ej. enviar un SMS).
     *
     * @param Usuario $usuario El usuario que intenta iniciar sesión.
     * @return bool True si el desafío se envió con éxito.
     */
    public function enviarDesafio(Usuario $usuario): bool
    {
        // TODO: Implementar la lógica de envío de SMS
    }

    /**
     * Verifica si el código proporcionado por el usuario es válido.
     *
     * @param Usuario $usuario El usuario.
     * @param string $codigo El código 2FA (ej. "123456").
     * @return bool True si el código es válido.
     */
    public function verificarCodigo(Usuario $usuario, string $codigo): bool
    {
        // TODO: Implementar la lógica de verificación de SMS
    }
}
?>

```

Si intentáramos guardar esta clase sin los métodos `enviarDesafio` o `verificarCodigo`, PHP nos daría un error fatal. La interfaz nos *obliga* a implementarlos.